



Indian Institute of Technology Delhi

Department of Electrical Engineering

Optimal Control Project

Project Title

Robot Path Planning Using Dynamic Programming and LQR

M.Tech in Control and Automation

Faculty: Prof. SAUNAK SEN

Name of Student: Akhilesh Kumar Patel

Entry Number: 2025EEA2725

Submission Date: 6 MAY 2026

Submitted to the Department of Electrical Engineering, IIT Delhi
as part of the M.Tech in Control and Automation Programme

Abstract

This project presents the design and implementation of an autonomous robot navigation framework using **Dynamic Programming (DP)** for path planning and **Linear Quadratic Regulator (LQR)** for path tracking and optimal control. The robot operates inside a two-dimensional grid-based environment containing free cells and obstacle cells. The objective is to generate an optimal collision-free path from a start point to a goal point and then make the robot follow that path smoothly in simulation.

The system is implemented in **ROS2 Humble** and simulated using **Gazebo 11** on Ubuntu 22.04. First, the environment is represented as a discrete grid world. Then, Dynamic Programming computes the minimum cost-to-go from every valid grid cell to the goal. Based on this cost map, an optimal path is extracted and published as a ROS topic. A differential-drive mobile robot modeled in URDF is spawned in Gazebo. The robot receives the path, estimates its current state through odometry, and follows the planned path using velocity commands. To improve motion quality, an LQR-based tracking controller is implemented to minimize tracking error and control effort.

The project demonstrates the successful integration of mathematical planning, control theory, ROS communication, and robotic simulation. It provides a solid foundation for advanced navigation methods such as A*, MPC, dynamic obstacle avoidance, and real robot deployment.

Contents

1	Introduction	4
2	Objectives	4
3	Software and Tools Used	5
4	System Overview	5
5	System Block Diagram	5
6	Problem Formulation	6
7	Environment Representation	7
8	Dynamic Programming for Path Planning	7
8.1	Concept of Cost-to-Go	7
8.2	Bellman Optimality Equation	7
8.3	Obstacle Handling	8
8.4	Value Iteration Form	8
8.5	Policy Extraction	8
8.6	Interpretation	8
9	Dynamic Programming Algorithm	9
10	DP Results	9
10.1	DP Path Output for Different Obstacle Configurations	9
10.1.1	Test Case 1: Initial Obstacle Configuration	9
10.1.2	Test Case 2: Modified Obstacle Layout	9
10.1.3	Test Case 3: Dense Obstacle Environment	10
10.1.4	Overall Analysis	10
11	Robot Kinematic Model	10
11.1	Meaning of the Kinematic Model	11
12	State-Space Model	11
13	Basic Path Follower	12
14	LQR Control Theory	12
14.1	Motivation for LQR	12
14.2	Error State	12
14.3	LQR Performance Index	13
14.4	Interpretation of Q and R	13
14.5	Riccati Equation	13
14.6	Optimal Gain Matrix	13
14.7	Optimal Control Law	13
14.8	Practical Use in This Project	14

15 ROS2 Architecture	14
15.1 Nodes	14
16 Code Implementation	14
16.1 Dynamic Programming Planner (dp_planner.py)	14
16.2 Basic Path Follower (path_follower.py)	15
16.3 LQR Controller (lqr_follower.py)	16
16.4 Robot Model (simple_robot.urdf)	16
16.5 Topics	17
16.6 Project Flowchart	18
17 Gazebo Simulation	18
17.1 Robot Model	18
17.2 Gazebo Plugins	19
17.3 Gazebo Robot View	19
17.4 Gazebo Terminal Output	19
18 Results and Discussion	19
18.1 Observed Results	20
18.2 Discussion	20
19 Advantages	20
20 Limitations	21
21 Future Scope	21
22 Conclusion	21
23 References	21

1 Introduction

Autonomous navigation is one of the most important and fundamental problems in mobile robotics. A robot operating in a real or simulated environment must perform three basic tasks successfully:

1. **Understand the environment** by representing free regions and obstacle regions.
2. **Plan a path** from the starting position to the target position.
3. **Track the path** using suitable control inputs while maintaining stability and smoothness.

In robotics, these tasks are usually studied under environment representation, planning, and control. In this project, all these parts are integrated into one complete pipeline. A grid environment is used to model the workspace. Dynamic Programming is used to compute an optimal obstacle-free path. After obtaining the path, a differential-drive robot is simulated in Gazebo and controlled in ROS2. Initially, a simple path follower is used, and then a more advanced LQR-based controller is introduced to improve the quality of motion.

The significance of this project lies in the fact that it combines planning and control in a practical ROS2-Gazebo workflow. It is not only a theoretical exercise but also a simulation-based implementation of a robot navigation system.

2 Objectives

The objectives of this project are:

- To create a grid-based environment containing free cells and obstacle cells.
- To define the robot start point and goal point.
- To compute the optimal path using Dynamic Programming.
- To generate a cost map for the environment.
- To publish the path in ROS2 as a navigation message.
- To model and simulate a differential-drive robot in Gazebo.
- To make the robot follow the planned path using velocity commands.
- To implement an LQR controller for better trajectory tracking.
- To evaluate the integrated performance of planning and control in simulation.

3 Software and Tools Used

Tool / Software	Purpose
Ubuntu 22.04	Operating system used for development and execution
ROS2 Humble	Middleware for communication between nodes
Gazebo 11	Robot simulation platform
Python	Implementation language for planning and control nodes
URDF	Robot modeling format
Dynamic Programming	Global path planning method
LQR	Optimal path tracking controller
Odometry	Robot state feedback

Table 1: Software and tools used in the project

4 System Overview

The overall system consists of the following stages:

1. Environment creation in the form of a grid world.
2. Obstacle placement.
3. Dynamic Programming-based cost computation.
4. Path extraction from start to goal.
5. Path publishing in ROS2.
6. Robot spawning in Gazebo.
7. Robot localization using odometry.
8. Path tracking using a controller.
9. LQR-based improvement in tracking performance.

5 System Block Diagram

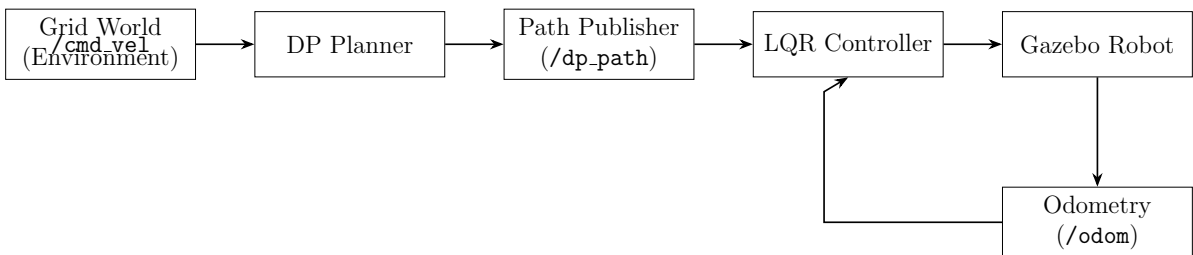


Figure 1: Block diagram of robot path planning and control system

Explanation:

The system begins with a grid-based environment where obstacles are defined. The Dynamic Programming planner computes the optimal cost map and generates the shortest path. This path is published as a ROS2 topic.

The LQR controller subscribes to both the planned path and odometry feedback, computes the control input, and sends velocity commands to the robot via the `/cmd_vel` topic. The robot operates in the Gazebo simulation environment, and its state is continuously fed back through odometry, forming a closed-loop control system.

6 Problem Formulation

The robot moves in a discrete two-dimensional environment. Let the set of all possible cells be represented as:

$$\mathcal{S} = \{(i, j) \mid i, j \in \mathbb{Z}, 0 \leq i \leq 9, 0 \leq j \leq 9\}$$

where:

- i denotes the row index
- j denotes the column index

The obstacle set is defined as:

$$\mathcal{O} \subset \mathcal{S}$$

The start state is:

$$s_0 = (0, 0)$$

The goal state is:

$$s_g = (9, 9)$$

The objective is to find a path

$$\pi = \{s_0, s_1, s_2, \dots, s_g\}$$

such that:

$$\pi \cap \mathcal{O} = \emptyset$$

and the total cost is minimized.

This formulation means that the path must be collision-free and optimal according to the chosen movement cost.

7 Environment Representation

The environment is modeled as a binary occupancy grid. Each cell belongs to one of the following two categories:

$$G(i, j) = \begin{cases} 0, & \text{if the cell is free} \\ 1, & \text{if the cell is an obstacle} \end{cases}$$

The grid size used in this project is:

$$10 \times 10$$

The robot can move in four possible directions:

$$A = \{\text{up, down, left, right}\}$$

This is also known as 4-connected motion.

The use of a grid world simplifies the planning problem because the state space becomes finite and discrete. Each state corresponds to one cell of the environment, and obstacle cells are simply marked as invalid.

8 Dynamic Programming for Path Planning

8.1 Concept of Cost-to-Go

Dynamic Programming solves the path planning problem by assigning to each state a value that represents the minimum cost required to reach the goal from that state. This value is called the **cost-to-go**.

Let $J(s)$ denote the minimum cost-to-go for state s . If the robot is already at the goal, then:

$$J(s_g) = 0$$

For all other states, the cost is determined recursively using the Bellman optimality principle.

8.2 Bellman Optimality Equation

The Bellman equation for this problem is:

$$J(s) = \min_{a \in A} [c(s, a) + J(s')]$$

where:

- s is the current state
- a is an action
- s' is the next state after applying action a
- $c(s, a)$ is the immediate step cost

In this project, each valid movement has unit cost:

$$c(s, a) = 1$$

Therefore, the equation becomes:

$$J(s) = \min_{a \in A} [1 + J(s')]$$

This means that the cost of a state is equal to one plus the minimum cost among all valid neighboring cells.

8.3 Obstacle Handling

If a state belongs to the obstacle set, it is never allowed in the path. Hence obstacle cells are treated as blocked and are not updated during cost propagation. Their cost can be viewed as:

$$J(s) = \infty, \quad \forall s \in \mathcal{O}$$

This guarantees that the optimal path avoids obstacle cells.

8.4 Value Iteration Form

The iterative update rule can be written as:

$$J_{k+1}(s) = \min_{a \in A} [1 + J_k(s')]$$

where k is the iteration number. The algorithm stops when the cost values stop changing.

8.5 Policy Extraction

Once the final cost map is obtained, the optimal policy is extracted as:

$$\pi(s) = \arg \min_{a \in A} J(s')$$

This means that from the current state, the robot selects the next neighboring state with the minimum cost value.

8.6 Interpretation

The DP planner does not directly search for one path only. Instead, it computes the best possible cost for *every* reachable state. That is why the method is robust and mathematically elegant. After the full cost map is available, the path is simply obtained by descending along the minimum-cost direction.

9 Dynamic Programming Algorithm

The complete DP algorithm used in this project is:

1. Initialize all free cells with very large cost.
2. Set goal cell cost equal to zero.
3. Mark obstacle cells as blocked.
4. For each free cell, examine all valid neighboring cells.
5. Update its cost using the Bellman equation.
6. Repeat the update process until convergence.
7. Start from the start cell.
8. Move to the neighboring free cell with minimum cost.
9. Continue until the goal is reached.

This gives both the cost map and the final obstacle-free path.

10 DP Results

10.1 DP Path Output for Different Obstacle Configurations

10.1.1 Test Case 1: Initial Obstacle Configuration

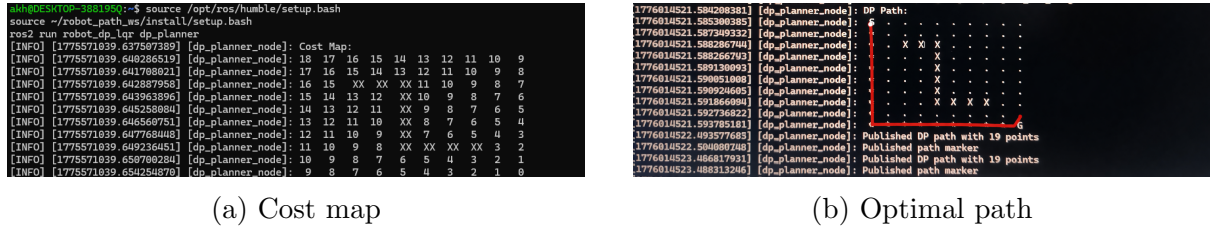


Figure 2: DP result for initial obstacle configuration

Observation: The cost map smoothly decreases toward the goal, and the extracted path follows the minimum cost trajectory while avoiding obstacles.

10.1.2 Test Case 2: Modified Obstacle Layout

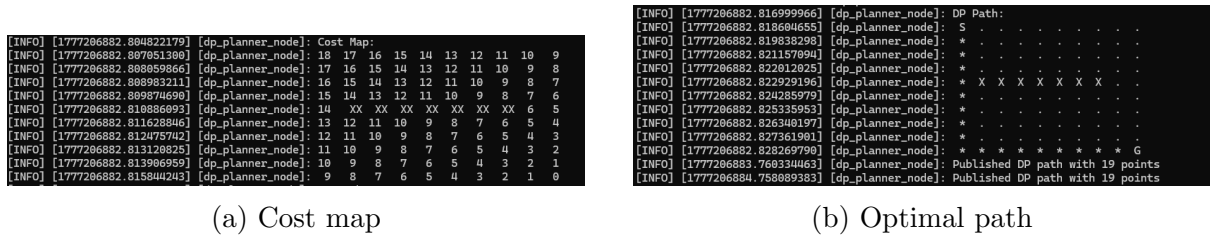


Figure 3: DP result for modified obstacle configuration

Observation: The planner recomputes the cost-to-go values and generates a new path that adapts to the changed obstacle layout, maintaining optimality.

10.1.3 Test Case 3: Dense Obstacle Environment

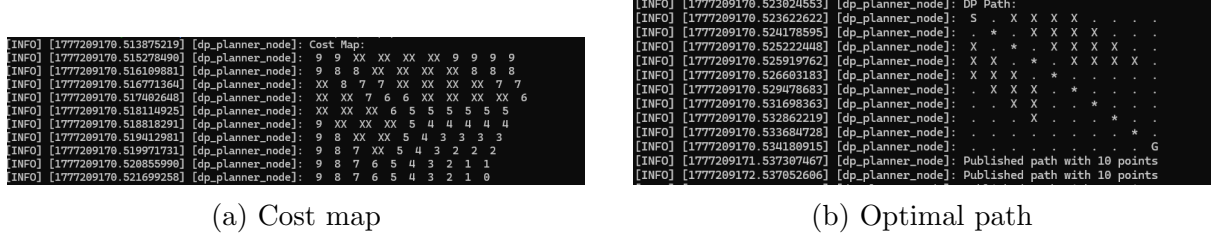


Figure 4: DP result for dense obstacle configuration

Observation: Even in a complex environment, the DP algorithm finds a feasible path. The trajectory becomes longer and more intricate due to constraints.

10.1.4 Overall Analysis

- DP computes optimal paths in all configurations.
- Cost maps correctly represent distance-to-go for each environment.
- Path adapts dynamically based on obstacle placement.
- Higher obstacle density increases path complexity and computation effort.

Conclusion: The DP-based path planner consistently generates globally optimal and collision-free paths across different obstacle scenarios.

Path Extraction Principle The path is obtained by starting from the initial state and iteratively selecting the neighboring cell with the minimum cost value. Since the cost map is generated using dynamic programming, this guarantees that the resulting trajectory is globally optimal.

11 Robot Kinematic Model

The simulated robot is a differential-drive robot. Its continuous-time kinematic equations are:

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \omega$$

where:

- x = robot position in the x-direction
- y = robot position in the y-direction
- θ = robot heading angle
- v = linear velocity
- ω = angular velocity

These equations show that the translational motion depends on both the linear velocity and the heading angle, while the rotational motion depends on the angular velocity.

11.1 Meaning of the Kinematic Model

The first two equations describe how the robot moves in the plane. If the robot points in the direction θ , then its linear velocity v is projected into x and y components using cosine and sine. The third equation updates the robot heading based on angular velocity.

12 State-Space Model

For controller design, the robot state vector is written as:

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

and the control input vector is:

$$\mathbf{u} = \begin{bmatrix} v \\ \omega \end{bmatrix}$$

The nonlinear robot model can be expressed as:

$$\dot{\mathbf{x}} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix}$$

For LQR design, this nonlinear model is linearized around a reference trajectory or operating point and represented in the standard state-space form:

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$$

where:

- A is the state matrix
- B is the input matrix

This linearized model is suitable for optimal regulator design.

13 Basic Path Follower

Before applying LQR, a simple path-following controller is implemented. The controller selects the next waypoint from the path and computes positional and heading errors.

Let the current robot pose be (x, y, θ) , and let the current target waypoint be (x_t, y_t) . Then the error in x and y is:

$$e_x = x_t - x$$

$$e_y = y_t - y$$

The Euclidean distance to the target waypoint is:

$$d = \sqrt{(x_t - x)^2 + (y_t - y)^2}$$

The desired heading toward the target is:

$$\theta_t = \tan^{-1} \left(\frac{y_t - y}{x_t - x} \right)$$

The heading error is:

$$e_\theta = \theta_t - \theta$$

This controller works reasonably well, but it can produce abrupt corrections and oscillations, especially at sharp corners or when the heading error is large.

14 LQR Control Theory

14.1 Motivation for LQR

The LQR controller is used to improve path tracking performance. Instead of only reacting to immediate distance and angle errors, LQR computes a control input that optimally balances:

- reduction in state error
- reduction in unnecessary control effort

This gives smoother and more stable robot motion.

14.2 Error State

The error state is defined between the actual robot pose and the reference pose:

$$\mathbf{e} = \begin{bmatrix} x - x_{\text{ref}} \\ y - y_{\text{ref}} \\ \theta - \theta_{\text{ref}} \end{bmatrix}$$

where:

- $(x_{\text{ref}}, y_{\text{ref}})$ are the desired reference coordinates
- θ_{ref} is the desired reference heading

The goal of the controller is to drive $\mathbf{e}$ toward zero.

14.3 LQR Performance Index

The quadratic cost function minimized by LQR is:

$$J = \int_0^\infty (\mathbf{x}^T Q \mathbf{x} + \mathbf{u}^T R \mathbf{u}) dt$$

where:

- Q is the state weighting matrix
- R is the input weighting matrix

14.4 Interpretation of Q and R

The matrix Q determines how strongly state errors are penalized. If the entries of Q are large, the controller tries harder to reduce those state errors.

The matrix R determines how strongly control effort is penalized. If the entries of R are large, the controller avoids very large control inputs and tries to move more smoothly.

Thus, Q and R together define the trade-off between tracking accuracy and smoothness.

14.5 Riccati Equation

To solve the LQR problem, one must solve the continuous-time algebraic Riccati equation:

$$A^T P + P A - P B R^{-1} B^T P + Q = 0$$

where P is the unknown positive semi-definite matrix.

14.6 Optimal Gain Matrix

After solving the Riccati equation, the optimal gain matrix is obtained as:

$$K = R^{-1} B^T P$$

14.7 Optimal Control Law

The feedback control law is:

$$\mathbf{u} = -K \mathbf{x}$$

or, in terms of error dynamics,

$$\mathbf{u} = -K \mathbf{e}$$

This means the control input is directly computed from the weighted state error.

14.8 Practical Use in This Project

In the implemented LQR follower:

- the next path point acts as the reference
- odometry provides the current robot state
- the tracking error is computed
- the LQR gain is used to generate linear and angular velocity commands

This produces smoother path tracking than the basic controller.

15 ROS2 Architecture

The project is divided into multiple ROS2 nodes.

15.1 Nodes

The system is implemented using multiple ROS2 nodes, each responsible for a specific task:

- `grid_world.py`: Defines the grid-based environment and obstacle layout.
- `dp_planner.py`: Computes the cost map using Dynamic Programming and publishes the optimal path.
- `path_follower.py`: Implements a basic waypoint-following controller.
- `lqr_follower.py`: Implements an optimal Linear Quadratic Regulator (LQR)-based tracking controller.

16 Code Implementation

This section presents the core implementation of the system including the Dynamic Programming planner, LQR controller, and robot model. Only the essential parts of the code are shown here for clarity, while the complete code can be referred to in the appendix.

16.1 Dynamic Programming Planner (`dp_planner.py`)

The DP planner computes the cost-to-go for each grid cell and extracts the optimal path.

```
def compute_cost(self):
    self.cost = [[math.inf]*self.cols for _ in range(self.rows)]
    self.cost[self.goal[0]][self.goal[1]] = 0

    changed = True
    while changed:
        changed = False
        for r in range(self.rows):
```

```

    for c in range(self.cols):
        if self.grid[r][c] == 1:
            continue
        for dr,dc in self.moves:
            nr,nc = r+dr,c+dc
            if 0<=nr<self.rows and 0<=nc<self.cols:
                if self.cost[r][c] > 1 + self.cost[nr][nc]:
                    self.cost[r][c] = 1 + self.cost[nr][nc]
                    changed = True

```

The optimal path is then extracted by moving toward the minimum-cost neighbor:

```

def extract_path(self):
    path = []
    current = self.start
    path.append(current)

    while current != self.goal:
        r,c = current
        best = current
        best_cost = self.cost[r][c]

        for dr,dc in self.moves:
            nr,nc = r+dr,c+dc
            if self.cost[nr][nc] < best_cost:
                best = (nr,nc)
                best_cost = self.cost[nr][nc]

        current = best
        path.append(current)

    return path

```

16.2 Basic Path Follower (path_follower.py)

The basic controller moves the robot toward successive waypoints.

```

dx = target.x - self.x
dy = target.y - self.y

distance = math.sqrt(dx*dx + dy*dy)

if distance > 0.1:
    cmd.linear.x = 0.4
    cmd.angular.z = math.atan2(dy, dx)
else:
    self.index += 1

```

This method is simple but can produce oscillations and abrupt turns.

16.3 LQR Controller (lqr_follower.py)

The LQR controller improves trajectory tracking by minimizing both state error and control effort.

The performance index is:

$$J = \int_0^{\infty} (x^T Q x + u^T R u) dt$$

The optimal control law is:

$$u = -Kx$$

The implementation computes control input as:

```
dx = target.x - self.x
dy = target.y - self.y

state = np.array([[dx], [dy], [theta]])

u = -K @ state

cmd.linear.x = float(u[0])
cmd.angular.z = float(u[1])
```

This results in smoother and more stable robot motion compared to the basic controller.

16.4 Robot Model (simple_robot.urdf)

The robot is modeled as a differential-drive system using URDF. The key components include:

- Base link
- Two wheels (left and right)
- Continuous joints
- Differential drive plugin

```
<plugin name="diff_drive" filename="libgazebo_ros_diff_drive.so">
  <left_joint>left_wheel_joint</left_joint>
  <right_joint>right_wheel_joint</right_joint>
  <wheel_separation>0.32</wheel_separation>
  <wheel_diameter>0.1</wheel_diameter>
  <command_topic>cmd_vel</command_topic>
  <odometry_topic>odom</odometry_topic>
</plugin>
```

This plugin enables velocity-based control and odometry feedback in Gazebo.

16.5 Topics

Communication between different components is achieved using ROS2 topics:

- `/dp_path`: Path published by the DP planner.
- `/odom`: Provides robot odometry (position and orientation).
- `/cmd_vel`: Velocity commands sent to the robot.
- `/tf`: Transform frames used for coordinate transformations.

16.6 Project Flowchart

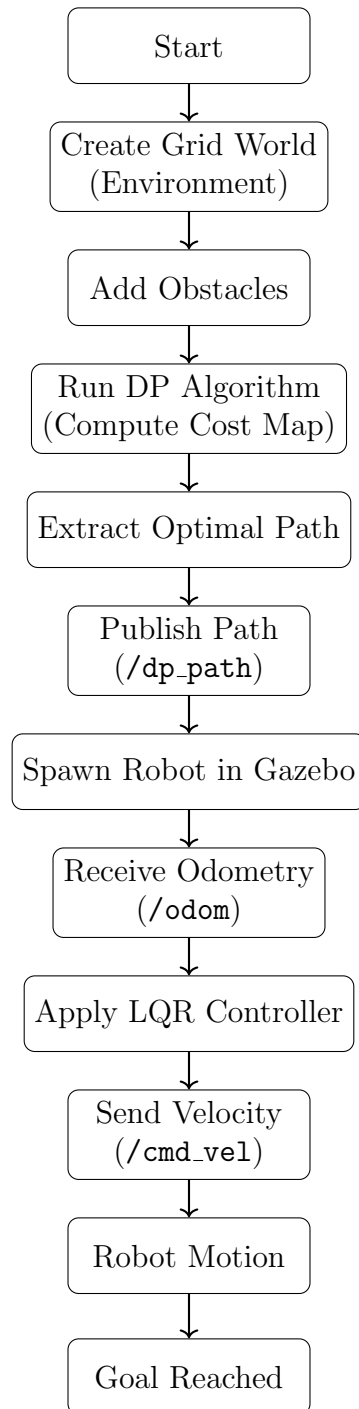


Figure 5: Flowchart of the complete project execution

17 Gazebo Simulation

17.1 Robot Model

The robot is modeled using URDF and consists of the following components:

- Base link

- Left wheel
- Right wheel
- Differential drive plugin

17.2 Gazebo Plugins

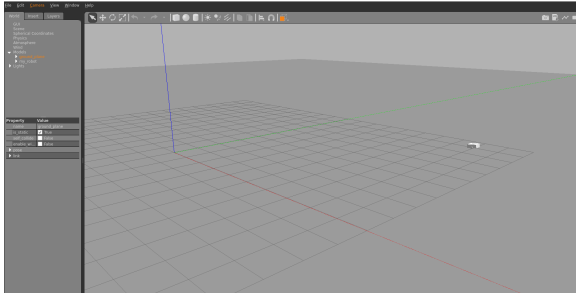
The following Gazebo plugins are used:

- `libgazebo_ros_init.so`
- `libgazebo_ros_factory.so`
- `libgazebo_ros_diff_drive.so`

These plugins enable:

- ROS2 initialization within Gazebo
- Model spawning in the simulation environment
- Subscription to velocity commands through `/cmd_vel`
- Publishing of odometry data through `/odom`

17.3 Gazebo Robot View



(a) Gazebo Robot View

```

1776014521.584288381 [dp_planner_node]: DP Path:
1776014521.585300385 [dp_planner_node]:
1776014521.587340932 [dp_planner_node]:
1776014521.588286744 [dp_planner_node]:
1776014521.588266793 [dp_planner_node]:
1776014521.589130093 [dp_planner_node]:
1776014521.590051908 [dp_planner_node]:
1776014521.590924605 [dp_planner_node]:
1776014521.591866094 [dp_planner_node]:
1776014521.592726822 [dp_planner_node]:
1776014521.593785181 [dp_planner_node]:
1776014522.493577683 [dp_planner_node]: Published DP path with 19 points
1776014522.506020740 [dp_planner_node]: Published path markers
1776014523.486817931 [dp_planner_node]: Published DP path with 19 points
1776014523.488313246 [dp_planner_node]: Published path marker
  
```

(b) Gazebo Terminal Output

Figure 6: Gazebo simulation environment showing robot visualization and terminal interaction

17.4 Gazebo Terminal Output

18 Results and Discussion

The obtained results show that the complete navigation framework works successfully.

```

akh@DESKTOP-388195Q:~$ source /opt/ros/humble/setup.bash
source ~/robot_path_ws/install/setup.bash
ros2 run robot_dp_lqr lqr_follower
[INFO] [1776101031.308063149] [lqr_follower]: Goal reached.
[INFO] [1776101092.068826276] [lqr_follower]: Goal reached.
[INFO] [1776101155.593373083] [lqr_follower]: Goal reached.
[INFO] [1776101216.482013372] [lqr_follower]: Goal reached.
[INFO] [1776101280.895648243] [lqr_follower]: Goal reached.
[INFO] [1776101344.615420000] [lqr_follower]: Goal reached.
[INFO] [1776101405.482837958] [lqr_follower]: Goal reached.
[INFO] [1776101465.883988108] [lqr_follower]: Goal reached.
[INFO] [1776101526.086378697] [lqr_follower]: Goal reached.
[INFO] [1776101589.587380918] [lqr_follower]: Goal reached.

```

Figure 7: Terminal output showing odometry and command interface in Gazebo

18.1 Observed Results

- The planner generated a valid and obstacle-free path.
- The cost map was correctly computed.
- The path was published on the ROS topic.
- The robot was successfully spawned in Gazebo.
- Odometry feedback was received properly.
- The robot followed the path successfully.
- LQR provided smoother turns and better stability compared to the simple path follower.

18.2 Discussion

The project clearly shows the benefit of separating planning and control. Dynamic Programming provides a mathematically optimal route in the discrete environment. LQR improves the tracking quality by regulating both the state error and control effort. The ROS2-based modular structure makes the system easier to debug, extend, and test.

19 Advantages

The major advantages of this approach are:

- Global path optimality in a static grid world
- Proper obstacle avoidance
- Smooth control through LQR
- Clean modular architecture using ROS2
- Realistic simulation in Gazebo

20 Limitations

The current implementation also has some limitations:

- Static obstacles only
- Grid-based path may be jagged
- LQR depends on a linearized model
- No uncertainty or sensor noise handling
- No real-time replanning

21 Future Scope

The project can be extended in several directions:

- dynamic obstacle avoidance
- A* or D* path planning
- MPC-based tracking
- spline-based trajectory smoothing
- RViz visualization
- implementation on a real mobile robot

22 Conclusion

This project successfully implements a complete autonomous navigation framework using Dynamic Programming for path planning and Linear Quadratic Regulator for trajectory tracking in ROS2 and Gazebo. The environment is represented as a grid with obstacles, the planner computes an optimal collision-free path, and the robot follows the path using feedback control. The Dynamic Programming approach provides a strong mathematical basis for optimal path selection in a static environment, while the LQR controller improves motion quality by minimizing tracking error and control effort.

The final outcome is a practically useful and academically strong robotics project that integrates mathematical modeling, planning theory, control theory, ROS communication, and Gazebo simulation in one complete system.

23 References

1. R. Bellman, *Dynamic Programming*, Princeton University Press, 1957.
2. D. E. Kirk, *Optimal Control Theory: An Introduction*, Dover Publications, 2004.
3. H. Kwakernaak and R. Sivan, *Linear Optimal Control Systems*, Wiley-Interscience, 1972.

4. A. Bhatt, A. Maity, and K. Das, “Path Planning for UAV using Sniff-Dog-Algorithm,” in *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, 2018, pp. 692–701.
5. S. P. Marin, “Optimal parametrization of curves for robot trajectory design,” *IEEE Transactions on Automatic Control*, vol. 33, no. 2, pp. 209–214, 1988.
6. K. Shin and N. McKay, “Robust trajectory planning for robotic manipulators under payload uncertainties,” *IEEE Transactions on Automatic Control*, vol. 32, no. 12, pp. 1044–1054, 1987.
7. H. Geering, L. Guzzella, S. Hepner, and C. Onder, “Time-optimal motions of robots in assembly tasks,” *IEEE Transactions on Automatic Control*, vol. 31, no. 6, pp. 512–518, 1986.
8. ROS 2 Documentation, <https://docs.ros.org>
9. Gazebo Documentation, <https://gazebo.org>

THANK YOU